

Production-Ready Repository Enhancements

To maximize the real-world resilience, automation, and usability of the **Pennsylvania Health Insurance Audit 2026** repository, the following production components are integrated directly into the codebase: containerization configuration, a continuous integration pipeline, a lightweight data API, and an automated PDF generation script.

1. Containerization (Dockerfile and docker-compose.yml)

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies for PDF generation and build tools
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpango-1.0-0 \
    libharfbuzz0b \
    libpangoft2-1.0-0 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "src/tools/api.py"]
```

docker-compose.yml

```
version: '3.8'

services:
  audit-api:
    build: .
    container_name: pa-health-audit-2026-api
    ports:
      - "8000:8000"
    volumes:
      - ./assets:/app/assets
      - ./deliverables:/app/deliverables
```

```
environment:
  - PORT=8000
  - ENVIRONMENT=production
restart: unless-stopped
```

requirements.txt

```
pandas==2.2.1
fastapi==0.110.0
uvicorn==0.28.0
reportlab==4.1.0
requests==2.31.0
pytest==8.1.0
```

2. Continuous Integration Workflow (.github/workflows/ci.yml)

```
name: Audit Data Pipeline CI
```

```
on:
```

```
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

```
jobs:
```

```
  validate-data:
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Checkout repository
 uses: actions/checkout@v4
- name: Set up Python 3.11
 uses: actions/setup-python@v5
 with:
 python-version: '3.11'
 cache: 'pip'
- name: Install dependencies
 run: |
 python -m pip install --upgrade pip
 pip install -r requirements.txt
- name: Run data validation scripts
 run: |
 python src/tools/vega-lite-validator.py

```
python src/tools/citation-generator.py

- name: Run test suite
  run: |
    pytest tests/
```

3. API Endpoints for Data Access (src/tools/api.py)

```
from fastapi import FastAPI, HTTPException
import json
import os

app = FastAPI(
    title="Pennsylvania Health Insurance Audit 2026 API",
    version="1.0.0",
    description="Programmatic access to audit datasets, enrollment
timelines, and county termination metrics."
)

DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__),
"../../assets/data"))

def load_json_file(filename: str):
    filepath = os.path.join(DATA_DIR, filename)
    if not os.path.exists(filepath):
        raise HTTPException(status_code=404, detail=f"Dataset
{filename} not found.")
    with open(filepath, "r", encoding="utf-8") as f:
        return json.load(f)

@app.get("/")
def read_root():
    return {
        "project": "Pennsylvania Health Insurance Audit 2026",
        "endpoints": [
            "/api/v1/enrollment",
            "/api/v1/demographics",
            "/api/v1/premiums",
            "/api/v1/peer-states"
        ]
    }

@app.get("/api/v1/enrollment")
def get_enrollment():
    return load_json_file("enrollment-timeline.json")

@app.get("/api/v1/demographics")
```

```

def get_demographics():
    return load_json_file("demographic-breakdowns.json")

@app.get("/api/v1/premiums")
def get_premiums():
    return load_json_file("premium-increase-data.json")

@app.get("/api/v1/peer-states")
def get_peer_states():
    return load_json_file("peer-states-comparison.json")

if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", 8000))
    uvicorn.run(app, host="0.0.0.0", port=port)

```

4. Automated PDF Generation Script (src/tools/generate_pdf.py)

```

import os
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer,
Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors

def
generate_audit_pdf(output_path="deliverables/audit-report/summary-repo
rt.pdf"):
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    doc = SimpleDocTemplate(
        output_path,
        pagesize=letter,
        rightMargin=54,
        leftMargin=54,
        topMargin=54,
        bottomMargin=54
    )

    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        'ReportTitle',
        parent=styles['Heading1'],
        fontSize=22,
        leading=26,
        textColor=colors.HexColor("#1A365D"),
        spaceAfter=12
    )

```

```

)

heading_style = ParagraphStyle(
    'ReportHeading',
    parent=styles['Heading2'],
    fontSize=14,
    leading=18,
    textColor=colors.HexColor("#2B6CB0"),
    spaceBefore=12,
    spaceAfter=6
)

body_style = ParagraphStyle(
    'ReportBody',
    parent=styles['Normal'],
    fontSize=10,
    leading=14,
    textColor=colors.HexColor("#2D3748"),
    spaceAfter=8
)

elements = []

# Title & Header
elements.append(Paragraph("Pennsylvania Health Insurance Audit
2026", title_style))
elements.append(Paragraph("Executive Summary & Impact Analysis",
heading_style))
elements.append(Paragraph(
    "The expiration of federal Enhanced Premium Tax Credits (EPTC)
on December 31, 2025, "
    "resulted in an immediate coverage crisis across Pennsylvania.
This document summarizes key metrics "
    "synthesized from 22 verified source documents.",
    body_style
))

# Statistics Table
data = [
    ["Metric", "Value"],
    ["Coverage Lost (Jan-Jul 2026)", "177,000+"],
    ["PA Average Premium Increase", "102%"],
    ["National Average Premium Increase", "58%"],
    ["Federal Subsidy Lost", "$600M/year"],
    ["State Replacement Appropriated", "$0"],
    ["Proposed State Investment", "$50M"]
]

```

```

t = Table(data, colWidths=[250, 254])
t.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor("#2B6CB0")),
    ('TEXTCOLOR', (0,0), (-1,0), colors.whitesmoke),
    ('ALIGN', (0,0), (-1,-1), 'LEFT'),
    ('FONTNAME', (0,0), (-1,0), 'Helvetica-Bold'),
    ('FONTSIZE', (0,0), (-1,0), 10),
    ('BOTTOMPADDING', (0,0), (-1,0), 6),
    ('BACKGROUND', (0,1), (-1,-1), colors.HexColor("#F7FAFC")),
    ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor("#CBD5E0")),
    ('FONTNAME', (0,1), (-1,-1), 'Helvetica'),
    ('FONTSIZE', (0,1), (-1,-1), 9),
    ('TOPPADDING', (0,1), (-1,-1), 6),
    ('BOTTOMPADDING', (0,1), (-1,-1), 6),
]))

elements.append(Spacer(1, 10))
elements.append(t)

doc.build(elements)
print(f"Successfully generated PDF report at {output_path}")

if __name__ == "__main__":
    generate_audit_pdf()

```